1. Introduction

Après avoir défini la vision globale, le périmètre du Produit Minimum Viable (MVP) et l'architecture technique générale du backend de Quantara, cette étape cruciale se concentre sur l'organisation concrète des fichiers et des répertoires. L'objectif est de fournir une feuille de route claire et sans ambiguïté pour l'implémentation, en répondant précisément aux questions : **quels fichiers existent, à quoi sert chacun, et quel contenu doit y être placé ou absolument évité ?**

Une structure de projet bien définie est fondamentale pour garantir la propreté du code, faciliter la collaboration, accélérer le développement et assurer la maintenabilité à long terme. Elle sert de guide opérationnel pour tout développeur, évitant le chaos et la dispersion de la logique applicative.

2. Objectif de cette Étape

L'objectif principal est d'établir une structure backend **opérationnelle et intuitive**. Cette clarté permettra à tout membre de l'équipe de développement de localiser et de placer correctement le code, répondant ainsi à des questions fondamentales telles que :

- Où implémenter une nouvelle route API ?
- Où réside la logique métier spécifique à un domaine ?
- Où définir un modèle de base de données ou un schéma de validation ?
- Où intégrer un fournisseur de données externe ?

Cette étape transforme l'architecture conceptuelle en une **structure de travail concrète et exploitable**.

3. Structure Racine du Backend

La base du projet backend est organisée comme suit :

```
backend/
├── pyproject.toml      # Configuration du projet Python et de ses outils
├── requirements.txt    # Liste des dépendances Python
├── alembic.ini         # Configuration du système de migrations de base de
données
├── app/                # Répertoire principal de l'application FastAPI
│   ├── main.py         # Point d'entrée de l'application
│   ├── core/           # Configurations et composants transversaux
│   ├── db/             # Gestion de la base de données
│   ├── api/            # Définition des routes API
│   ├── modules/        # Modules métier de l'application
│   ├── shared/         # Code partagé et utilitaires transversaux
│   └── tests/          # Tests unitaires et d'intégration
└── scripts/            # Scripts d'administration et d'initialisation
```

4. Fichiers Racine du Backend

4.1 `pyproject.toml`

Rôle : Fichier de configuration moderne pour les projets Python (PEP 518/621). Il centralise les métadonnées du projet, les dépendances de construction et la configuration d'outils de développement (linters, formateurs, test runners). **À mettre dedans :** Métadonnées du projet, dépendances principales, configuration de `ruff`, `black`, `pytest`, etc. **À ne pas mettre dedans :** Logique applicative, secrets, configuration runtime spécifique au backend.

4.2 `requirements.txt`

Rôle : Liste explicite des dépendances Python requises pour le projet, facilitant l'installation et la reproductibilité de l'environnement. **À mettre dedans :** `FastAPI`, `Uvicorn`, `SQLAlchemy`, `Alembic`, `Pydantic`, `psycopg` (driver PostgreSQL), `passlib` / `bcrypt` (pour le hachage des mots de passe), `python-jose` (pour JWT), `pytest`. **À ne pas mettre dedans :** Commentaires métier, logique technique, informations de structure.

4.3 `alembic.ini`

Rôle : Fichier de configuration pour Alembic, l'outil de migrations de base de données. Il spécifie les paramètres de connexion à la base de données et le chemin vers les scripts de migration. **À mettre dedans :** Paramètres liés aux migrations, chemin du script Alembic. **À ne pas mettre dedans :** Logique de base de données, code SQLAlchemy, settings métier.

5. Le Dossier `app/`

Ce répertoire est le cœur de l'application backend, regroupant tous les composants de l'application FastAPI.

5.1 `app/main.py`

Rôle : Point d'entrée principal de l'application FastAPI. Il initialise l'application, configure les middlewares et agrège les routeurs des différentes API. **Il doit contenir :** Création de l'instance `FastAPI`, configuration globale de l'application (CORS, middlewares), enregistrement du routeur principal (`/api/v1`), et éventuellement des gestionnaires d'erreurs globaux. **Il ne doit pas contenir :** Logique métier, requêtes SQLAlchemy, calculs de simulation, logique d'authentification détaillée, ou de gros blocs conditionnels spécifiques à un module. **Résumé :** `main.py` doit rester **léger et centralisateur**, agissant comme un orchestrateur de haut niveau.

6. Le Dossier `app/core/`

Ce dossier contient tout ce qui est **global au backend** et transversal, mais qui n'appartient pas à un module métier particulier. Il assure la cohérence et la centralisation des configurations essentielles.

```
app/core/
├── config.py       # Gestion des configurations et variables d'environnement
├── security.py     # Fonctions de sécurité (hachage, tokens)
├── logging.py      # Configuration du système de logs
└── exceptions.py   # Définition des exceptions applicatives globales
```

6.1 `config.py`

Rôle : Gérer la configuration centrale du backend, typiquement via des variables d'environnement et des settings Pydantic. **À mettre dedans :** `DATABASE_URL`, `JWT_SECRET_KEY`, `ACCESS_TOKEN_EXPIRE_MINUTES`, `COINGECKO_BASE_URL`, flags d'environnement (debug, prod). **À ne pas mettre dedans :** Logique métier, appels DB, routes, fonctions lourdes.

6.2 `security.py`

Rôle : Centraliser la logique de sécurité réutilisable à travers l'application. **À mettre dedans :** Fonctions de hachage et vérification de mots de passe (`hash_password()`, `verify_password()`), création et vérification de tokens (`create_access_token()`), helpers d'authentification. **À ne pas mettre dedans :** Requêtes DB d'utilisateurs, endpoints de connexion, logique business complète.

6.3 `logging.py`

Rôle : Configurer le système de journalisation (logging) du backend. **À mettre dedans :** Format des logs, niveau de logs, handlers de base. **À ne pas mettre dedans :** Logique métier, gestion des exceptions applicatives dispersée.

6.4 `exceptions.py`

Rôle : Définir les exceptions applicatives communes et les erreurs de domaine, permettant une gestion d'erreurs cohérente. **À mettre dedans :** Exceptions custom (`ResourceNotFoundError`, `UnauthorizedError`, `InvalidSimulationOrderError`), mapping d'erreurs globales. **À ne pas mettre dedans :** Logique complète de réponse HTTP dans tous les modules, exceptions trop spécifiques à un seul module si elles n'y servent que localement.

7. Le Dossier `app/db/`

Ce dossier regroupe tous les éléments liés à la gestion de la base de données.

```
app/db/
├── base.py         # Déclaration de la base ORM commune
├── session.py      # Configuration de l'engine et des sessions SQLAlchemy
└── migrations/     # Fichiers de migrations Alembic
```

7.1 `base.py`

Rôle : Déclaration de la base ORM commune pour SQLAlchemy, souvent `Base = declarative_base()`. **À mettre dedans :** La déclaration de base et éventuellement les imports agrégés des modèles pour Alembic. **À ne pas mettre dedans :** Logique métier, sessions, requêtes.

7.2 `session.py`

Rôle : Configuration de l'engine et des sessions SQLAlchemy, ainsi que la dépendance pour l'injection de session dans FastAPI. **À mettre dedans :** `SessionLocal`, `engine`, `get_db()` (dépendance pour FastAPI). **À ne pas mettre dedans :** SQL spécifique aux modules, logique d'authentification, calculs métier.

7.3 `migrations/`

Rôle : Contient les scripts de migrations de base de données générés et gérés par Alembic. **À mettre dedans :** Versions de migrations, scripts de migration auto/manuels. **À ne pas mettre dedans :** Logique runtime, modèles métiers, requêtes applicatives.

8. Le Dossier `app/api/`

Ce dossier est dédié à la définition des routes HTTP de l'API, organisées par version et par domaine fonctionnel.

```
app/api/
├── deps.py         # Dépendances réutilisables pour les routes
└── v1/             # Version 1 de l'API
    ├── router.py  # Routeur principal de la v1
    ├── auth.py    # Routes d'authentification
    ├── assets.py  # Routes pour les actifs de marché
    ├── charting.py# Routes pour les données de graphique
    ├── simulation.py # Routes pour la simulation de trading
    ├── education.py  # Routes pour les modules éducatifs
    ├── progress.py   # Routes pour le suivi de progression
    └── insights.py   # Routes pour les insights (minimal au MVP)
```

8.1 `deps.py`

Rôle : Centraliser les dépendances réutilisables par les routes FastAPI. **À mettre dedans :** `get_db` (injection de session DB), `get_current_user`, `get_current_active_user` (dépendances d'authentification), dépendances de permissions. **À ne pas mettre dedans :** Logique métier, routes, requêtes complexes.

8.2 `v1/router.py`

Rôle : Routeur principal de la version 1 de l'API. Il agrège tous les routeurs spécifiques aux domaines fonctionnels. **À mettre dedans :** Agrégation de tous les routeurs via `include_router(...)` pour organiser l'API v1. **À ne pas mettre dedans :** Logique métier, handlers complexes, code DB.

8.3 Fichiers de Routes par Domaine (ex: `auth.py`, `assets.py`)

Rôle : Chaque fichier gère les endpoints REST pour un domaine fonctionnel spécifique. Ils reçoivent les requêtes HTTP, valident les entrées/sorties et appellent les services métier correspondants. **Chaque fichier de route doit contenir :** Endpoints REST, dépendances FastAPI, validation entrée/sortie (via Pydantic), appel des services métier. **Il ne doit pas contenir :** Requêtes DB directes, logique complète de calcul, règles métier lourdes. **Résumé :** La route doit être **fine** : `recevoir → appeler service → renvoyer réponse`.

9. Le Dossier `app/modules/`

Ce dossier est le cœur métier du backend, où la logique applicative est découpée en modules autonomes, chacun représentant un domaine fonctionnel de Quantara.

```
app/modules/
├── identity/       # Gestion des utilisateurs et de l'authentification
├── market_data/    # Données de marché et intégration des fournisseurs
├── charting/       # Préparation des données pour les graphiques
├── simulation/     # Moteur de simulation de trading
├── education/      # Contenu et progression pédagogique
├── progress/       # Suivi de la progression utilisateur
└── insights/       # Moteur d'analyse et de recommandations
```

10. Structure Standard d'un Module

Chaque module au sein de `app/modules/` suit une structure interne homogène, garantissant la cohérence et la prévisibilité.

```
modules/simulation/
├── models.py       # Modèles SQLAlchemy spécifiques au module
├── schemas.py      # Schémas Pydantic pour les échanges API du module
├── repository.py   # Logique d'accès aux données du module
├── service.py      # Logique métier principale du module
└── rules.py        # Règles métier pures et calculs du module
```

10.1 `models.py`

Rôle : Définir les entités persistées en base de données pour le module. **À mettre dedans :** Tables SQLAlchemy, colonnes, relations, contraintes simples, index. **Exemple `simulation` :** `Portfolio`, `Position`, `Trade`, `TransactionHistory`. **À ne pas mettre dedans :** Calcul de P&L, logique buy/sell, règles de validation métier, logique de service.

10.2 `schemas.py`

Rôle : Définir les schémas Pydantic pour les contrats d'entrée/sortie de l'API du module. **À mettre dedans :** Request schemas, response schemas, validation entrée/sortie, DTOs (Data Transfer Objects). **Exemple `simulation` :** `CreateTradeRequest`, `TradeResponse`, `PortfolioSummaryResponse`. **À ne pas mettre dedans :** Requêtes DB, logique métier, calculs techniques lourds.

10.3 `repository.py`

Rôle : Encapsuler l'accès aux données du module, en faisant abstraction de la base de données sous-jacente. **À mettre dedans :** Fonctions de lecture, écriture, requêtes filtrées, opérations CRUD, requêtes spécialisées via SQLAlchemy. **Exemples :**

`get_user_by_email()`, `get_open_positions()`, `list_courses()`. **À ne pas mettre dedans :** Logique métier, décisions produit, calculs métier lourds, validation business.

10.4 `service.py`

Rôle : Contenir la logique métier principale du module. Il orchestre les opérations, coordonne les repositories, appelle les règles et prend les décisions du domaine. **À mettre dedans :** Orchestration métier, coordination de repositories, appels aux règles, décisions du domaine, flux applicatif. **Exemples :** `login_user()`, `execute_trade()`, `mark_lesson_completed()`, `calculate_portfolio_summary()`. **À ne pas mettre dedans :** Routes FastAPI, SQL direct, configuration globale.

10.5 `rules.py`

Rôle : Définir les règles métier pures et les calculs réutilisables, indépendamment de l'infrastructure. **À mettre dedans :** Calculs, validations métier, règles de simulation, logique pure et testable. **Exemples :** `calculate_units()`, `validate_trade_amount()`, `compute_pnl()`, `validate_stop_loss()`. **À ne pas mettre dedans :** Accès DB, HTTP, dépendances FastAPI, sérialisation API.

11. Détail Module par Module

11.1 `identity/`

Rôle : Gérer tout ce qui concerne l'utilisateur et l'accès à la plateforme. **Fichiers attendus :** `models.py` (User), `schemas.py` (login, register, user response), `repository.py` (user lookup, create user), `service.py` (auth logic), `rules.py` (validations auth). **Il gère :** Utilisateurs, login, enregistrement (plus tard), hachage de mots de passe (via `core/security`), gestion des tokens (plus tard).

11.2 `market_data/`

Rôle : Gérer les données de marché, leur récupération, normalisation et distribution. **Fichiers attendus :** `models.py` (si persistance minimale d'actifs ou snapshots), `schemas.py` (asset response, candle response), `repository.py` (cache/lecture éventuelle), `service.py` (orchestration data provider), `rules.py` (normalisation simple), `providers/` (intégrations externes). **Sous-dossier recommandé :** `providers/` avec `base.py` (interface) et `coingecko.py` (implémentation spécifique). **Il gère :** Liste des actifs, détail d'un actif, données de bougies (candles), normalisation.

11.3 `charting/`

Rôle : Préparer et servir les données nécessaires à l'affichage des graphiques. **Fichiers attendus :** `schemas.py`, `service.py`, `indicators.py` (pour les calculs d'indicateurs techniques). **Pourquoi `indicators.py` ? :** Les indicateurs techniques sont une responsabilité spécifique qui mérite son propre fichier pour la clarté et la testabilité. **Il gère :** Timeframes, agrégation de données de graphique, calculs d'indicateurs (RSI/SMA plus tard).

11.4 `simulation/`

Rôle : Module central de Quantara, gérant toute la logique de simulation de trading. **Fichiers attendus :** `models.py`, `schemas.py`, `repository.py`, `service.py`, `rules.py`. **Il gère :** Trades simulés (buy/sell), positions, portefeuille, historique, calcul de P&L. **Note :** C'est le module qui doit être le plus rigoureux en termes de séparation des préoccupations et de testabilité.

11.5 `education/`

Rôle : Gérer le contenu pédagogique de la plateforme. **Fichiers attendus :** `models.py` (Course, Lesson), `schemas.py` (course/lesson response), `repository.py`, `service.py`. **Il gère :** Cours, leçons, détail de cours, détail de leçon.

11.6 `progress/`

Rôle : Suivre la progression des utilisateurs dans les modules éducatifs et les simulations. **Fichiers attendus :** `models.py` (CourseCompletion, UserStats), `schemas.py` (progress response), `repository.py`, `service.py`. **Il gère :** Complétion de cours, agrégation de statistiques, progression globale, données pour la page Progress.

11.7 `insights/`

Rôle : Moteur d'analyse comportementale et de recommandations (à développer après le MVP). **Au MVP :** Peut être très léger, contenant seulement `schemas.py`, `service.py`, `rules.py`. **Il gère (plus tard) :** Détection d'erreurs récurrentes, recommandations personnalisées, analyse des patterns utilisateur.

12. Le Dossier `app/shared/`

Ce dossier contient le code transversal partagé qui n'appartient pas spécifiquement à un module métier. Il est crucial de maintenir ce dossier propre et de n'y placer que des éléments véritablement réutilisables à travers plusieurs modules.

```
app/shared/
├── enums.py        # Définition des énumérations communes
├── types.py        # Types partagés ou alias globaux
├── utils.py        # Fonctions utilitaires transversales
└── responses.py    # Structures de réponses API standardisées
```

12.1 `enums.py`

Rôle : Définir les énumérations communes utilisées dans plusieurs modules. **Exemples :** `OrderSide` (BUY/SELL), `AssetClass` (CRYPTO/FOREX), `LessonStatus` (COMPLETED/IN_PROGRESS), `ProgressState`.

12.2 `types.py`

Rôle : Définir les types partagés ou les alias de types globaux pour améliorer la lisibilité et la cohérence du code.

12.3 `utils.py`

Rôle : Regrouper les fonctions utilitaires vraiment transversales et réutilisables. Il est impératif de veiller à ce que ce fichier ne devienne pas un “fourre-tout” de fonctions diverses et variées.

12.4 `responses.py`

Rôle : Définir des structures de réponses API standardisées (ex: `SuccessResponse`, `PaginatedResponse`, `ErrorResponse`) pour assurer une cohérence dans les retours d’API.

13. Le Dossier `app/tests/`

Ce dossier contient l’ensemble des tests pour le backend, organisés pour refléter la structure modulaire de l’application.

```
app/tests/
├── unit/           # Tests unitaires
├── integration/    # Tests d'intégration
└── conftest.py     # Fixtures pytest partagées
```

13.1 `unit/`

Rôle : Contient les tests unitaires, qui se concentrent sur la validation de petites unités de code isolées. **À mettre dedans :** Tests des `rules.py`, des `service.py` (logique

pure), des fonctions de calcul et des validations.

13.2 `integration/`

Rôle : Contient les tests d'intégration, qui vérifient l'interaction correcte entre plusieurs composants. **À mettre dedans :** Tests des endpoints API, des interactions avec la base de données, et des workflows complets (ex: login, exécution de trade, complétion de cours).

13.3 `conftest.py`

Rôle : Fichier de configuration pour pytest, permettant de définir des fixtures partagées à travers tous les tests. **Exemples :** Fixtures pour une session de base de données de test, un utilisateur factice, un client de test FastAPI, des données initiales (seeded data).

14. Le Dossier `scripts/`

Rôle : Regroupe les scripts techniques pratiques qui ne font pas partie du runtime de l'API, mais sont utiles pour le développement, l'administration ou la maintenance. **Exemples :** Scripts pour *seeder* des données initiales, importer des données de cours, initialiser l'environnement local, ou des scripts de maintenance ponctuels. **À ne pas mettre dedans :** Logique métier centrale, code API utilisé à chaque requête.

15. Règles de Placement du Code

Pour maintenir la clarté et la cohérence de l'architecture, voici des règles strictes pour le placement du code :

- **Si c'est une route HTTP :** `app/api/v1/...`
- **Si c'est une entité de base de données :** `models.py` (dans le module concerné)

- **Si c'est un schéma de requête/réponse API :** `schemas.py` (dans le module concerné)
- **Si c'est une requête d'accès à la base de données :** `repository.py` (dans le module concerné)
- **Si c'est la logique métier principale :** `service.py` (dans le module concerné)
- **Si c'est une règle pure / un calcul métier :** `rules.py` (dans le module concerné)
- **Si c'est une intégration avec un service externe :** `providers/...` (dans le module `market_data` ou un module dédié si plus général)

16. Ce qu'il ne Faut Surtout Pas Faire

Pour préserver l'intégrité de l'architecture, certaines pratiques doivent être absolument évitées :

- **Mettre du SQL direct dans les routes ou les services :** L'accès à la base de données doit toujours passer par la couche Repository.
- **Mettre toute la logique dans `service.py` sans sous-structure :** Les services doivent orchestrer, les règles doivent calculer.
- **Faire un `utils.py` géant :** Le dossier `shared/utils.py` doit rester léger et contenir uniquement des fonctions véritablement transversales.
- **Mélanger l'authentification, la simulation et les données de marché dans les mêmes fichiers :** Chaque module doit être autonome.
- **Coder tout dans `main.py` :** Ce fichier est un point d'entrée, pas un monolithe.
- **Appeler les providers externes directement depuis le frontend :** Toutes les interactions externes doivent transiter par le backend pour des raisons de sécurité, de contrôle et de normalisation.

17. Conclusion

Cette étape a permis de verrouiller l'organisation détaillée des fichiers et des répertoires du backend de Quantara. En définissant précisément le rôle de chaque

dossier et fichier, la structure standard d'un module, et les règles de placement du code, nous avons créé une base **directement exploitable** pour l'implémentation.

Cette clarté architecturale est essentielle pour guider les développeurs, faciliter la documentation et assurer que le backend de Quantara sera construit de manière robuste, maintenable et évolutive. Nous disposons désormais d'une feuille de route concrète pour passer à l'implémentation des premières routes API et des fonctionnalités du MVP.